Mattermost Cross Guard CDS Whitepaper

Architecture, deployment, and security analysis for cross-domain message relay

1. Executive Summary

Cross Guard is a Mattermost plugin that bridges isolated Mattermost instances across security domain boundaries using one of four transport providers as the relay layer: NATS (pub/sub), Azure Queue Storage (polling), Azure Blob Storage (batched write-ahead log), or Azure Service Bus (AMQP with PeekLock and dead-letter). It is designed for environments where servers cannot share databases, APIs, or network access, including air-gapped networks, multi-classification deployments, and multi-tenant architectures.

The plugin relays posts, edits, deletes, and reactions bidirectionally while keeping each Mattermost server fully independent. No database synchronization, no direct API calls between servers, and no shared authentication. Each server maintains its own users, teams, channels, and data store. Cross Guard operates purely at the message level, publishing serialized envelopes (JSON or XML) onto the configured transport (a NATS subject, an Azure Storage queue, a batched WAL blob in an Azure Blob container, or an Azure Service Bus queue) and consuming them on the other side.

Administrative controls enforce explicit opt-in at both the team and channel level. No messages flow until an administrator deliberately links a connection to a team and then to specific channels within that team. Inbound connections require interactive acceptance via admin prompts before any data crosses the boundary.

Transport security supports TLS 1.2+ across all providers and provider-specific authentication: NATS supports mutual certificate authentication (mTLS), token or credentials authentication, and ACLs for subject-level access control; Azure Queue and Azure Blob use either Azure Storage shared-key credentials or an Azure AD Service Principal (client-credentials flow) over TLS-always endpoints; Azure Service Bus uses either a SAS connection string (with SAS material sanitized out of error logs, and scoped SAS rules supported over the namespace-wide root key) or an Azure AD Service Principal. Service Principal mode allows credentials to be scoped via Azure RBAC and rotated in Azure AD without touching plugin config; admin guidance for SP setup, the minimum required data-plane roles, and the Federal-grade pattern for keeping the `client_secret` out of plugin config (Mattermost env-var substitution of the

connections JSON) are documented in the admin guide. Permission lockdown is configurable via the `RestrictToSystemAdmins` setting.

This whitepaper provides a comprehensive technical analysis of Cross Guard's architecture, message lifecycle, security model, and deployment topologies for audiences evaluating the plugin for cross-domain use.

2. The Cross Domain Problem

What is a Cross Domain Solution?

A Cross Domain Solution (CDS) is a system that enables controlled information transfer between two or more security domains that operate at different classification levels or under different administrative authorities. In government and defense contexts, these domains are often separated by policy, regulation, or physical network segmentation. A CDS must provide a controlled, auditable, and secure pathway for information to cross these boundaries without violating the isolation guarantees of either domain.

Traditional CDS approaches include hardware guards (dedicated appliances that inspect and filter data at the network boundary), data diodes (one-way physical connections that enforce unidirectional flow), and protocol breaks (systems that terminate one protocol and re-originate data in another, preventing direct network connectivity).

These approaches work well for file transfers, email, and structured data exchange. However, they were not designed for real-time collaborative messaging. Modern collaboration tools like Mattermost operate within a single domain: users on Domain A cannot see or interact with messages on Domain B. There is no native mechanism for bridging conversations across isolated instances.

Why Existing Federation Approaches Fall Short

Several approaches exist for connecting separate Mattermost instances, but each has fundamental limitations for cross-domain use:

Approach	Mechanism	Why It Fails for CDS
Shared Database	Multiple Mattermost instances read/write from the same database	Requires direct network access between domains to the database server. Violates network isolation. The database becomes a single point of compromise spanning both domains.
Direct API Federation	Servers call each other's REST APIs to synchronize data	Requires bidirectional HTTP connectivity between servers. Violates network segmentation policies. Exposes the full Mattermost API surface across the domain boundary.
Email Bridges	Messages forwarded as emails between instances	Loses real-time conversation flow. No edit or delete synchronization. No reaction support. Latency measured in minutes, not milliseconds. Conversations become disjointed threads.
Webhook Relays	Outgoing webhooks trigger incoming webhooks on the other server	Requires direct HTTP connectivity (same problem as API federation). Point-to-point, fragile, no built-in resilience or reconnection. No standardized authentication beyond shared secrets.

What Cross Guard Provides

Cross Guard takes a fundamentally different approach. Rather than synchronizing databases, proxying APIs, or bridging protocols, it operates as a message-level relay with a decoupled pub/sub architecture:

- **Message-level relay, not database sync:** Only individual post events (create, edit, delete, react) cross the boundary. No bulk data transfer, no schema coupling, no shared state.
- **Asynchronous, decoupled architecture:** The two Mattermost servers never communicate directly. A transport intermediary (a NATS message bus, an Azure Storage account hosting a queue or a WAL blob container, or an Azure Service Bus namespace) sits between them. Publishers and subscribers interact only with the transport layer, not with each other.
- **Full server independence:** Each server maintains its own users, teams, channels, database, authentication, and administration. Cross Guard adds relay capability without modifying any of these.
- **Opt-in at every level:** Relay is not automatic. Administrators must explicitly link connections to teams, then to individual channels. Inbound connections require interactive

acceptance before any messages flow.

- **Minimal data crossing the boundary:** Only text content, routing metadata, and optionally file attachments transit the transport layer. File transfer is disabled by default and must be explicitly enabled per connection with configurable type filtering and size limits. User metadata, channel properties, post properties, and interactive message elements are intentionally excluded.

3. Transport Layer Options

Cross Guard supports four transport providers: NATS (pub/sub), Azure Queue Storage (polling-based), Azure Blob Storage (batched write-ahead log), and Azure Service Bus (AMQP with PeekLock and dead-letter). Administrators select a provider per connection in the System Console. All four providers carry the same serialized envelope format (JSON or XML) and support the same message types. The choice depends on infrastructure, latency requirements, durability needs, cost profile, and operational preferences.

NATS

What is NATS?

NATS is a lightweight, high-performance messaging system written in Go. It provides core pub/sub semantics: publishers send messages to named subjects, and subscribers receive messages from those subjects. NATS is designed for cloud-native, distributed systems and is used in production by organizations ranging from startups to government agencies.

For message relay, Cross Guard uses NATS core pub/sub, which is fire-and-forget: messages are delivered to connected subscribers in real time, with no message persistence or replay. For optional file transfer, Cross Guard uses NATS JetStream Object Store with a hardcoded bucket name of `crossguard-files` (not user-configurable, unlike Azure's `blob_container_name`). This maps directly to Cross Guard's relay model, where messages are relayed as they occur and not queued for later delivery.

Why NATS Fits the CDS Model

1. Decoupled Architecture

Publishers and subscribers never communicate directly. The NATS server mediates all traffic. Neither the publisher nor the subscriber needs to know the other's network address, identity,

or even existence. This maps directly to CDS architecture where the transport sits between domains and neither domain has direct access to the other.

2. Subject-Based Routing

Messages are published to named subjects (e.g., `crossguard.relay.a-to-b`). Subscribers filter by subject. This provides natural channel isolation: different relay directions use different subjects, and NATS ACLs can restrict which clients can publish or subscribe to which subjects. No routing configuration is needed beyond subject names.

3. Lightweight Protocol

NATS uses a text-based protocol with minimal overhead. A typical Cross Guard message envelope is a few hundred bytes (JSON or XML). NATS adds negligible framing. The result is sub-millisecond publish latency, which is critical for real-time messaging where users expect near-instant delivery.

4. TLS and Authentication

NATS has native support for TLS 1.2+, token authentication, username/password authentication, and client certificate authentication (mTLS). These map directly to CDS transport security requirements. No additional proxy or TLS terminator is needed.

5. Network Placement Flexibility

A NATS server can be deployed in a DMZ, an enclave, a guard network, or any controlled network segment between domains. The Mattermost servers only need to reach the NATS server; they never need to reach each other. This is the key architectural property that enables CDS deployment.

6. Resilience

NATS clients have built-in reconnection with configurable retry behavior. Cross Guard configures unlimited reconnects with a 2-second wait, so temporary network interruptions (maintenance windows, transient failures) self-heal without manual intervention. Disconnect and reconnect events are logged for operational visibility.

7. ACL Support

NATS server authorization rules can restrict which clients can publish or subscribe to which subjects. In a CDS deployment, this means the NATS administrator can enforce that Server A can only publish to `crossguard.a-to-b` and only subscribe to `crossguard.b-to-a`, and vice versa. This provides transport-level access control independent of the plugin's own authorization.

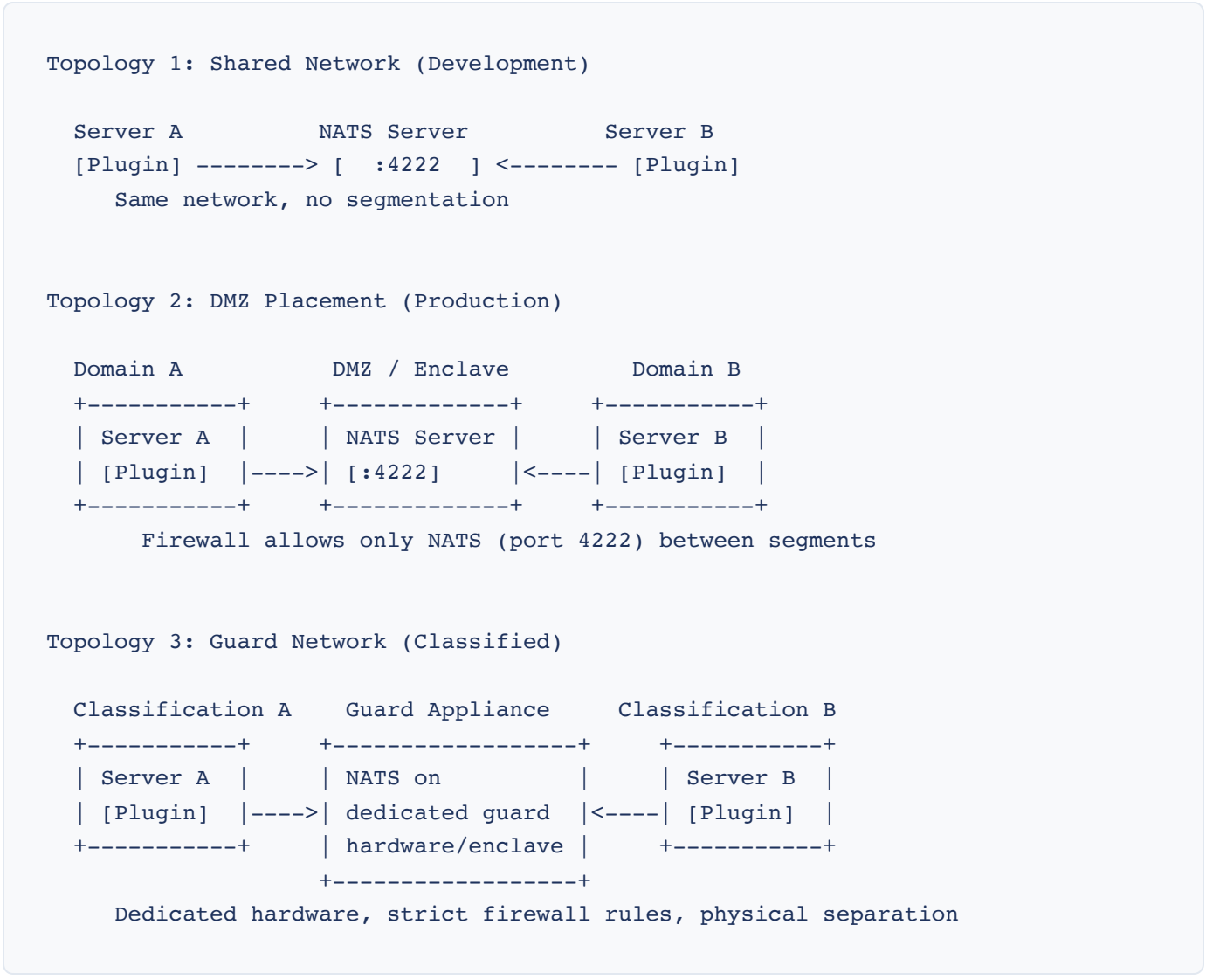
How Cross Guard Uses NATS

Each Cross Guard connection (inbound or outbound) maps to exactly one NATS subject. Outbound connections publish serialized message envelopes to their configured subject. Inbound connections subscribe to their configured subject and dispatch received messages to type-specific handlers.

The subject prefix `crossguard.` is enforced by configuration validation. All subjects must start with this prefix, which prevents accidental cross-contamination with other NATS traffic if the NATS server is shared (though a dedicated NATS server is recommended for CDS deployments).

Separate subjects per direction (e.g., `crossguard.a-to-b` and `crossguard.b-to-a`) prevent echo loops at the transport level. Even if the plugin's application-level loop prevention were to fail, messages published on one subject are not received on a different subject.

NATS Deployment Topologies for CDS



Azure Queue Storage

What is Azure Queue Storage?

Azure Queue Storage is a fully managed message queuing service provided by Microsoft Azure. It stores messages in queues that can be read by any client with a valid connection string or SAS token. Unlike NATS (which uses real-time pub/sub), Azure Queue Storage uses a polling model where consumers periodically check for new messages.

Why Azure Queue Storage Fits the CDS Model

1. Polling-Based Delivery

The plugin polls the Azure queue on a configurable interval (default 5 seconds for messages, 15 seconds for blob/file transfers). Messages are dequeued in batches of up to 32 with a 5-minute visibility timeout. This polling model is well-suited for environments where persistent connections are difficult to maintain or where store-and-forward semantics are preferred over real-time delivery.

2. At-Least-Once Delivery

Azure Queue Storage provides at-least-once delivery semantics. Messages remain in the queue until explicitly deleted after successful processing. If the consumer crashes before acknowledging, the message becomes visible again after the visibility timeout expires. This is more durable than NATS core pub/sub, which drops messages if no subscriber is connected.

3. Managed Service

Azure Queue Storage is a fully managed service. There is no NATS cluster to deploy, patch, monitor, or scale. The Azure Storage account handles availability, replication, and maintenance. This reduces operational overhead, especially in environments where infrastructure teams prefer managed services over self-hosted components.

4. Connection String Authentication

Authentication uses an Azure Storage connection string, which contains the account name and key. Mattermost does not encrypt plugin settings at rest, so the connection string is stored as plaintext in `config.json` and the Mattermost configuration database by default; for production, inject the connections JSON via the Mattermost env-var pattern instead (see Section 11: Secret Storage and Env-Var Injection). No separate TLS certificate management is required because all Azure Storage endpoints use TLS by default.

5. Auto-Provisioning

The plugin automatically creates the queue and blob container (as configured in `queue_name` and `blob_container_name`) on startup if they do not exist. No manual Azure resource

provisioning is needed beyond creating the Storage account itself.

How Cross Guard Uses Azure Queue Storage

Each outbound connection enqueues serialized message envelopes (JSON or XML) to the configured Azure queue. File attachments (when enabled) are uploaded to the configured blob container with a key in the format `{postId}/{randomId}`, referenced in the envelope. The inbound side polls the queue, processes each message, and deletes it after successful handling. For files, the inbound side downloads the blob and deletes it after processing.

Azure Blob Storage

What is Azure Blob Storage?

Azure Blob Storage is a fully managed object store. Cross Guard's Azure Blob transport does not use Azure Queue Storage at all. Instead of enqueuing one message per relay event, the outbound side appends events to an in-memory write-ahead log (WAL) and periodically seals and uploads the WAL as a single batch blob. The inbound side polls the container, takes a short-lived lease on each batch blob, processes its messages, and deletes the blob.

Why Azure Blob Storage Fits the CDS Model

1. Batched Writes Reduce API Cost

One batch blob carries many messages, so total Azure Storage API calls scale with the number of flush intervals rather than with the number of relay events. In high-volume channels this is significantly cheaper than per-message queue operations and easier to reason about for cost forecasting in regulated tenants.

2. Lease-Based Mutual Exclusion

Each batch blob is protected by a short-lived blob lease while an inbound poller processes it. If the poller crashes mid-flight, the lease expires after `blob_lock_max_age_seconds` (default 300) and another inbound node may break the lease and retry. This gives at-least-once delivery without an external coordinator.

3. Single-Container Footprint

Both batch WAL blobs and deferred file attachment blobs live in the same container. Operators only need to provision one container, which simplifies access policies, audit, and lifecycle rules in CDS environments.

4. Connection String Authentication

Authentication is the same Azure Storage connection string model as the Azure Queue provider. Mattermost does not encrypt plugin settings at rest, so the connection string is stored as plaintext in `config.json` and the Mattermost configuration database by default; inject via env vars for production (see Section 11: Secret Storage and Env-Var Injection). All Azure Blob traffic is over TLS.

How Cross Guard Uses Azure Blob Storage

Outbound buffers serialized envelopes in memory and every `flush_interval_seconds` (default 60, minimum 5) seals the buffer and uploads it as a time-ordered batch blob. Inbound lists the container on a poll cycle, acquires a lease on each batch blob, dispatches its messages to the inbound handler, then deletes the blob and releases the lease. File attachments are uploaded as deferred file blobs into the same container and reaped after delivery. Expected end-to-end latency is bounded by the configured flush interval.

Azure Service Bus

What is Azure Service Bus?

Azure Service Bus is a fully managed enterprise message broker speaking AMQP. It provides queue and topic semantics with first-class features that Azure Queue Storage lacks: PeekLock receive with explicit settle, server-side delivery counters, native dead-letter sub-queues, and configurable lock duration on the broker side.

Why Azure Service Bus Fits the CDS Model

1. PeekLock with Explicit Settle

Cross Guard receives messages under PeekLock, then calls Complete on handler success or Abandon on handler error. Service Bus increments `DeliveryCount` on every Abandon and, after the queue's `MaxDeliveryCount` is exceeded, automatically moves the message to the queue's dead-letter sub-queue. This gives operators poison-message containment without bespoke retry tracking.

2. Pre-Created Queues with ARM-Controlled Tunables

The plugin does *not* auto-create Service Bus queues. Operators provision the queue in the Azure portal, ARM, or Terraform with their chosen `LockDuration` and `MaxDeliveryCount`. Lock duration is an entity property and cannot be set from a client. This puts message-handling SLAs and DLQ policy under the same change-control as other Azure infrastructure.

3. SAS Connection String Authentication

Authentication is via a Service Bus SAS connection string of the form `Endpoint=sb://<namespace>.servicebus.windows.net/;SharedAccessKeyName=<rule>;SharedAccessKey=<key>`. Mattermost does not encrypt plugin settings at rest, so the SAS connection string is stored as plaintext in `config.json` and the Mattermost configuration database by default; for production, inject via env vars (see Section 11: Secret Storage and Env-Var Injection). Service Bus error messages that might leak SAS material are sanitized before logging.

4. Independent File Sidecar

File attachments are transferred via an Azure Blob Storage sidecar container with its own credentials, independent of the Service Bus namespace. Service Bus queues themselves remain message-only and never carry file bytes. This separation lets operators scale and audit each store independently.

How Cross Guard Uses Azure Service Bus

Each outbound connection sends a serialized envelope (JSON or XML) into the configured Service Bus queue via the Azure SDK. The inbound side calls `ReceiveMessages` in batches of up to 32 with a 5-second per-call wait, settles each message with Complete on success or Abandon on error, and logs a WARN when `DeliveryCount > 1` so operators can detect poison-message buildup before DLQ moves happen. The default outbound payload ceiling is 192,000 bytes (Service Bus Standard caps at 256 KiB per message; Premium supports up to 100 MiB). Inbound polls the optional blob sidecar every 15 seconds (configurable) for file attachments.

Transport Comparison

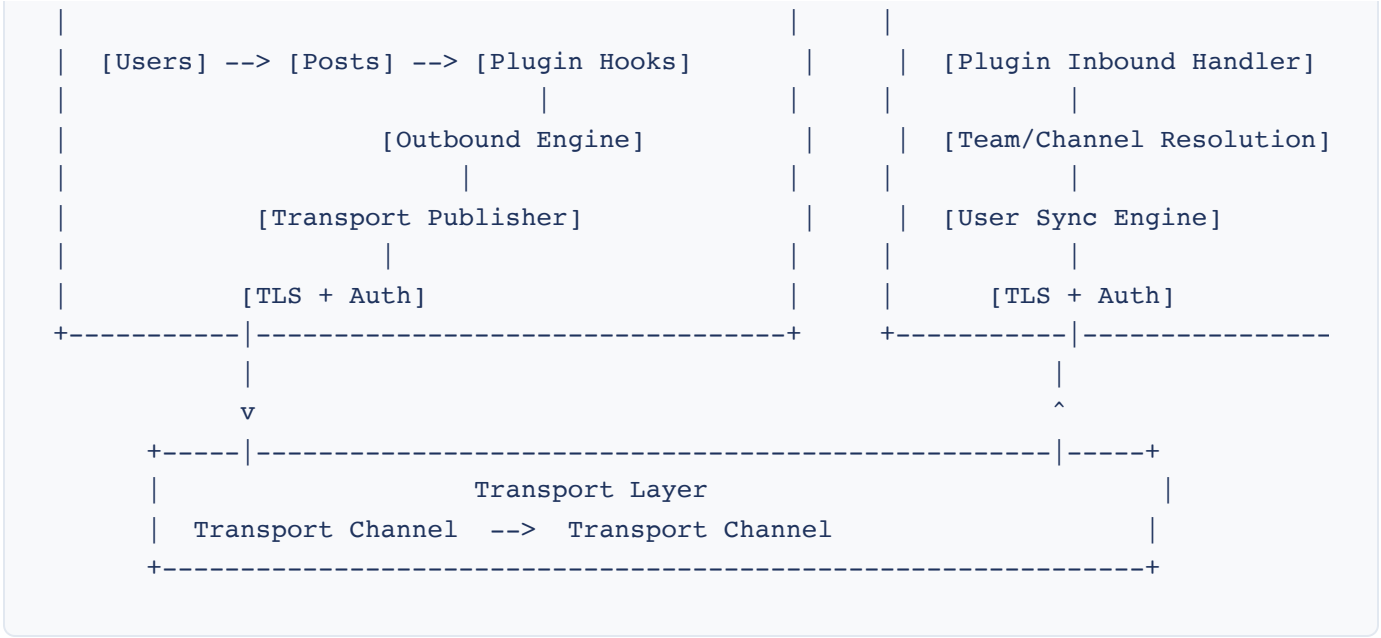
Aspect	NATS	Azure Queue Storage	Azure Blob Storage	Azure Service Bus
Latency	Sub-millisecond (pub/sub)	5-15 seconds (polling)	Bounded by flush interval (default 60s)	Bounded by receive max wait (5s) plus broker latency
Delivery	At-most-once	At-least-once	At-least-once	At-least-once (PeekLock with server-side DeliveryCount)
Message Size	1 MB default (NATS server	48,000 bytes (before	Bounded by batch blob	192,000 bytes default; 256 KiB Standard tier

Aspect	NATS	Azure Queue Storage	Azure Blob Storage	Azure Service Bus
	configurable)	Base64)	size (many messages per blob)	ceiling, 100 MiB on Premium
Infrastructure	Self-hosted NATS cluster	Azure managed service	Azure managed service	Azure managed service
Encryption	TLS (opt-in)	TLS always (Azure endpoint)	TLS always (Azure endpoint)	TLS always (AMQP over TLS)
File Transfer	JetStream Object Store (1h TTL)	Blob Storage (deleted after processing)	Same container as WAL (deferred file blobs)	Blob Storage sidecar (independent credentials)
Auto-Provisioning	No (NATS server must exist)	Queue and container auto-created	Container auto-created	No (queue must be pre-created with LockDuration and MaxDeliveryCount)
Best For	Low-latency, high-throughput environments	Azure-native environments, simplicity	High-volume channels where amortized API cost matters	Azure-native environments needing DLQ, redelivery counters, and ARM-controlled SLAs

4. System Architecture

Component Overview

```
Mattermost Server A                                    Mattermost Server B
+-----------------------------------------------+     +----------------------------
```



Note: The specific transport mechanism depends on the configured provider. For NATS, messages flow over a NATS subject (e.g., `crossguard.a-to-b`). For Azure Queue Storage, messages flow through an Azure Storage queue. For Azure Blob Storage, messages are appended to an in-memory WAL and flushed periodically as batched blobs into a shared container; the inbound side leases each blob, processes it, and deletes it. For Azure Service Bus, messages flow through a pre-created Service Bus queue using PeekLock receive with explicit Complete on success or Abandon on error.

Plugin Internal Architecture

The Cross Guard plugin is implemented as a single Go binary that runs inside the Mattermost server process. Its internal architecture consists of the following components:

- **`Plugin` struct:** The central struct that holds the bot user ID, KV store reference, transport provider connections (NATS, Azure Queue, Azure Blob, or Azure Service Bus), relay semaphore, wait group for goroutine lifecycle tracking, and a context for graceful shutdown.
- **Outbound pool:** A slice of `outboundConn` structs, each holding a `QueueProvider` (NATS connection, Azure Queue client, Azure Blob client with in-memory WAL buffer, or Azure Service Bus sender) and its configuration. Protected by a `sync.RWMutex` to allow concurrent reads (publishing) with exclusive writes (reconnection on config change).
- **Inbound pool:** A slice of `inboundConn` structs, each holding a `QueueProvider` (NATS subscription, Azure queue poller, Azure Blob lease-based reader, or Azure Service Bus

PeekLock receiver). Protected by a `sync.RWMutex` with the same read/write concurrency model.

- **Relay semaphore:** A buffered channel of capacity 256, shared by both inbound and outbound relay operations. Controls the maximum number of concurrent relay goroutines to prevent unbounded resource consumption under high message volume.
- **KV store:** Mattermost's built-in plugin key-value store, used for all persistent state (team/channel linkage, post mappings, prompt state, rewrite indexes). An optional caching layer wraps the KV store with 15-minute TTL and cluster-aware invalidation.

Plugin Lifecycle

Event	Actions
OnActivate	Creates bot user, initializes KV store and caching layer, registers the `/crossguard` slash command, initializes HTTP router for REST API and interactive prompts, creates context for shutdown signaling, spawns relay semaphore, connects all configured outbound and inbound connections (NATS, Azure Queue Storage, Azure Blob Storage, or Azure Service Bus depending on each connection's provider setting).
OnConfigurationChange	Validates new configuration, closes existing transport connections, opens new connections with updated settings. Live reconnection with no plugin restart required.
OnDeactivate	Cancels context (signals all goroutines to stop), closes inbound subscriptions (stops new message delivery), waits for all in-flight relay goroutines via WaitGroup, closes outbound connections.

5. Message Relay Lifecycle (Outbound)

This section traces the complete path of a message from the moment a user posts in a channel on Server A to the moment it is published to the transport layer (NATS, Azure Queue Storage, Azure Blob Storage, or Azure Service Bus).

Step 1: Post Lifecycle Hook Fires

Mattermost's plugin API provides lifecycle hooks that fire after post events. Cross Guard implements five hooks in `hooks.go`:

- `MessageHasBeenPosted()` for new posts
- `MessageHasBeenUpdated()` for edits
- `MessageHasBeenDeleted()` for deletes
- `ReactionHasBeenAdded()` for new reactions
- `ReactionHasBeenRemoved()` for reaction removals

Each hook receives the relevant post or reaction object from Mattermost and begins the relay evaluation pipeline.

Step 2: Filtering (What Does NOT Get Relayed)

Before any relay occurs, the hook applies a series of filters to determine whether the event should be relayed. Events that match any filter are silently skipped:

Filter	Check	Purpose
System messages	`post.IsSystemMessage()` returns true	Prevents system-generated messages (join/leave, header change, etc.) from relaying
Bot posts	`post.UserId == p.botUserID`	Prevents bot announcements (init, teardown, prompts) from relaying
Already-relayed posts	`post.GetProp("crossguard_relayed") != nil`	Primary loop prevention: stops inbound posts from being re-relayed outbound
Sync user reactions	`user.Position == "crossguard-sync"`	Prevents reactions added by sync users from being relayed back

Note: Not all filters apply to every hook. The system message and `crossguard_relayed` checks apply to post create and update hooks. The delete hook uses bot user ID and a deleting flag instead. Reaction hooks use bot user ID and sync-user position filtering.

Step 3: Channel and Team Validation

The `isChannelRelayEnabled()` function checks whether the channel and its parent team are linked to any outbound connections:

1. Look up the channel's linked connections from the KV store (key: `crossguard-channelinit-{channelID}`)
2. Look up the team's linked connections from the KV store (key: `crossguard-teaminit-{teamID}`)
3. Return the channel connections after verifying the team also has connections. The effective intersection happens at publish time via `isOutboundLinked()`.

If no connections match, the message is not relayed. This ensures that both team-level and channel-level opt-in are required.

Step 4: Envelope Construction

The hook fetches source metadata (user, channel, team) via the Plugin API and builds a type-specific payload:

Message Type	Payload Fields
`crossguard_post` / `crossguard_update`	`post_id`, `root_id`, `channel_id`, `channel_name`, `team_id`, `team_name`, `user_id`, `username`, `message`, `create_at`
`crossguard_delete`	`post_id`, `channel_id`, `channel_name`, `team_id`, `team_name`
`crossguard_reaction_add` / `crossguard_reaction_remove`	`post_id`, `channel_id`, `channel_name`, `team_id`, `team_name`, `user_id`, `username`, `emoji_name`

The payload is embedded in a `model.Envelope` struct with a type constant and an RFC 3339 UTC timestamp, then serialized via `model.Marshal()` to JSON or XML bytes depending on the connection's `message_format` setting.

Step 5: Semaphore Acquisition and Async Dispatch

The relay operation is dispatched asynchronously to avoid blocking the Mattermost post lifecycle:

1. Non-blocking attempt to acquire a slot from the relay semaphore (buffered channel of capacity 256)
2. If the semaphore is full (256 concurrent relays already in progress): drop the message and log a warning. This is a deliberate design choice to prevent unbounded goroutine spawning.

3. If a slot is acquired: increment the WaitGroup counter and spawn a goroutine for the publish operation

Step 6: Publish to Transport

Inside the goroutine, for each outbound connection whose name matches a linked connection from Step 3:

1. Check connection health via application-level tracking (success/failure of recent publishes, rechecked every 30 seconds). If unhealthy, skip the connection silently.
2. Call `provider.Publish(ctx, data)` with the serialized bytes (JSON or XML, per connection config)
3. Provider-specific behavior: **NATS** checks `IsConnected()` / `IsReconnecting()`, retries up to 3 times with exponential backoff (500ms, 1s, 2s), and flushes. **Azure Queue Storage** enqueues as Base64 without application-level retries (Azure SDK handles its own transport retries). **Azure Blob Storage** appends the serialized envelope to an in-memory write-ahead log buffer; the buffer is sealed and uploaded as a batch blob every `flush_interval_seconds` (default 60). **Azure Service Bus** sends the AMQP message via the Service Bus SDK and enforces a per-connection `max_message_size_bytes` (default 192,000) at the client boundary before sending; SAS-bearing error messages are sanitized before logging.
4. On failure: log an error and drop the message (no queuing)
5. Context cancellation is respected during backoff waits for graceful shutdown

After publishing (or dropping), the semaphore slot is released and the WaitGroup counter is decremented.

6. Message Relay Lifecycle (Inbound)

This section traces the complete path of a message from the moment it arrives via the transport layer (a NATS subject, an Azure Storage queue, a leased Azure Blob batch blob, or an Azure Service Bus queue) to the moment it appears as a local post on Server B.

Step 1: Transport Delivers Message

The `handleInboundMessage()` closure receives raw bytes (`[]byte`) from the transport provider: a NATS subscription callback, an Azure Queue Storage poller (5-second cadence, batches of up to

32), an Azure Blob Storage lease-based reader (one batch blob at a time, scoped by a short-lived blob lease), or an Azure Service Bus PeekLock receiver (batches of up to 32 with a 5-second max wait, Complete on success and Abandon on error). Like outbound, it performs a non-blocking semaphore acquisition from the same 256-slot pool. If the semaphore is full, the message is dropped with a warning log. If a slot is acquired, a goroutine is spawned.

Step 2: Envelope Deserialization

`model.DetectFormat(data)` auto-detects the wire format (JSON or XML) by inspecting the first non-whitespace byte, then `model.Unmarshal()` decodes the envelope. The handler dispatches by `envelope.Type`:

Type	Handler
`crossguard_post`	`handleInboundPost()`
`crossguard_update`	`handleInboundUpdate()`
`crossguard_delete`	`handleInboundDelete()`
`crossguard_reaction_add`	`handleInboundReaction(add=true)`
`crossguard_reaction_remove`	`handleInboundReaction(add=false)`
`crossguard_test`	Log and discard (connection testing)

Step 3: Team and Channel Resolution

The `resolveTeamAndChannel()` function maps the remote team and channel names to local entities:

1. **Check rewrite index first:** `GetTeamRewriteIndex(connName, remoteTeamName)` looks up KV key `crossguard-rwi-{connName}::{remoteTeamName}`. If a rewrite rule exists, use the local team ID from the index.
2. **If no rewrite:** `GetTeamByName(teamName)` assumes the remote and local team names match.
3. **Verify team linkage:** Check KV store (`crossguard-teaminit-{teamID}`) to confirm this team is linked to the inbound connection.
4. **If not linked:** Trigger `handleUnlinkedInbound()`, which posts an Accept/Block prompt to `#town-square`. The message is dropped (it will not be retried).

5. **Resolve channel:** `GetChannelByName(teamID, channelName)` to find the local channel.
6. **Verify channel linkage:** Check KV store for channel-level connection linkage.
7. **If channel not linked:** Trigger `handleUnlinkedInboundChannel()`, which posts a prompt in the channel itself. The message is dropped.

Step 4: User Resolution

The `resolveInboundUser()` function determines which local user will be the author of the relayed post. This is governed by the `UsernameLookup` setting:

- **If UsernameLookup enabled (default):** Call `GetUserByUsername(username)`. If a matching local user is found and their `Position` is not `"crossguard-sync"`, use the real local user. Messages appear as if that user posted them.
- **If not found or lookup disabled:** Create or fetch a synthetic "sync user" with:
  - Username: `{remoteUsername}.{connName}` (truncated to 64 characters)
  - Email: `crossguard.synthetic_user.{randomID}@crossguard.local`
  - Position: `crossguard-sync`
  - FirstName: remote username
  - LastName: `(via {connName})`
  - Props: `CrossguardRemoteUsername: {originalUsername}`
- **Race-safe creation:** Uses atomic create with retry on conflict. If two messages arrive simultaneously for the same unknown user, only one sync user is created.

After user resolution, `ensureMembership()` adds the user to the target team and channel if they are not already a member.

Step 5: Post Creation (New Posts)

1. Build a `mmModel.Post` with the resolved user ID, channel ID, and message text
2. If threaded (`RootID` present in envelope): look up the local root post via post mapping to preserve thread structure
3. Set `post.AddProp("crossguard_relayed", true)` to prevent re-relay (primary loop prevention)
4. Call `p.API.CreatePost(post)` to create the post locally
5. Store the mapping: `SetPostMapping(connName, remotePostID, localPostID)` for future edits/deletes

Step 5 (alt): Post Update

1. Look up local post ID via `GetPostMapping(connName, remotePostID)`
2. If no mapping exists: log a warning and skip (the original post may not have been relayed, e.g., if the channel was linked after the original was posted)
3. Fetch the local post, replace the `Message` field, call `UpdatePost()`

Step 5 (alt): Post Delete

1. Look up local post ID via post mapping
2. Set `SetDeletingFlag(localPostID)` in KV store (prevents the `MessageHasBeenDeleted` hook from re-relaying this delete)
3. Call `DeletePost(localPostID)`
4. Clear `ClearDeletingFlag(localPostID)`
5. Delete the post mapping entry

Step 5 (alt): Reaction Add/Remove

1. Look up local post ID via post mapping
2. Resolve user (same logic as for posts)
3. Call `AddReaction()` or `RemoveReaction()` with the local user ID and post ID

7. Data Boundary Control

Understanding exactly what data crosses the domain boundary is critical for CDS evaluation. This section documents every field in the transport envelope, its source, whether it is user-controlled, and how it is used on the receiving side. The envelope format is identical across all four transport providers (NATS, Azure Queue Storage, Azure Blob Storage, and Azure Service Bus).

Data That Crosses the Boundary (Transport Envelope)

Envelope Fields

The envelope is a single-layer structure (payloads are embedded directly, not serialized as strings). It may be serialized as JSON or XML depending on the outbound connection's `message_format`

setting.

Field	Content	User-Controlled?	Used For
`type`	Message type constant (e.g., `crossguard_post`)	No (set by plugin code)	Handler dispatch
`timestamp`	RFC 3339 UTC timestamp	No (set by plugin code)	Logging only
`post_message`	Post or update payload	Contains user-controlled fields (see below)	Create or update a post
`delete_message`	Delete payload	No (IDs only)	Delete a post
`reaction_message`	Reaction payload	Partially (emoji name is user-selected)	Add or remove a reaction
`test_message`	Test payload	No (generated ID)	Connectivity verification

Wire Format Encoding (JSON or XML)

Each outbound connection chooses its on-the-wire encoding via the `message_format` setting (`"json"` or `"xml"`). The envelope structure above is identical in both encodings; only the surface syntax differs. JSON uses snake_case field names (`post_message`); XML uses PascalCase elements rooted at `<CrossguardMessage xmlns="urn:mattermost-crossguard">` and validated against `schema/crossguard.xsd`. Inbound connections do not require format configuration: the parser inspects the first non-whitespace byte (`<` means XML, otherwise JSON) and strips a leading UTF-8 BOM if a CDS device prepended one.

The repository ships 12 schema-valid XML sample payloads under `schema/examples/`, one per message-type variant the plugin emits. Each file is byte-for-byte identical to `model.Marshal(env, FormatXML)` plus a trailing newline. The byte-equality contract is enforced by `TestExampleFiles_Symmetric` in `server/model/examples_roundtrip_test.go`, which also verifies that `Unmarshal` followed by `Marshal` is a fixed point and that marshalling is deterministic. Operators evaluating the protocol for CDS approval can point at these example files as the canonical wire shape and at the test as the evidence that the plugin will not silently drift away from them. Detailed JSON and XML field references live in transport-interface.html.

PostMessage Payload Fields

Field	Source	User-Controlled?	Used On Receive
`post_id`	`post.Id` (26-char Mattermost ID)	No	Post mapping key for edits/deletes
`root_id`	`post.RootId`	No	Thread parent resolution via post mapping
`channel_id`	`post.ChannelId`	No	Not used directly (name-based routing)
`channel_name`	`channel.Name`	Indirectly (admin-set)	Routing: `GetChannelByName()`
`team_id`	`channel.TeamId`	No	Not used directly
`team_name`	`team.Name`	Indirectly (admin-set)	Routing: `GetTeamByName()` or rewrite lookup
`user_id`	`post.UserId`	No	Not used on receive
`username`	`user.Username`	Yes (users choose usernames)	User resolution: `resolveInboundUser()`
`message`	`post.Message`	**Yes** (full post content)	Post body text
`create_at`	`post.CreateAt`	No	Not used on receive

DeleteMessage Payload Fields

Contains `post_id`, `channel_id`, `channel_name`, `team_id`, `team_name`. No user-controlled content beyond the admin-set team and channel names.

ReactionMessage Payload Fields

Contains `post_id`, `channel_id`, `channel_name`, `team_id`, `team_name`, `user_id`, `username` (user-controlled), and `emoji_name` (user-selected from Mattermost's emoji set).

Data Intentionally Excluded

Minimal data boundary
Cross Guard intentionally excludes the following from the relay envelope. This minimizes the attack surface and prevents accidental data spillage.

Excluded Data	Why Excluded
File attachments (`post.FileIds`)	File transfer is disabled by default. When enabled per connection, files are uploaded to a NATS JetStream Object Store (NATS provider) or Azure Blob Storage (Azure provider) and downloaded by the receiving side. For Azure Blob Storage, blobs are stored with the key format `{postId}/{randomId}`, where `postId` is the Mattermost post ID and `randomId` is a newly generated unique identifier. Administrators can restrict transfers using allow/deny file type filtering and a 100 MB size limit. Files may contain embedded metadata or classified content, so explicit opt-in and filtering are required.
Post properties (`post.Props`)	Props may contain plugin-specific data from other plugins (webhooks, integrations, custom metadata). Relaying them could leak internal system information.
User metadata (email, profile image, custom fields, roles, auth data)	User profiles may contain PII, classification markings, or organizational data that should not cross domain boundaries.
Channel metadata (purpose, header, type)	Channel headers may contain sensitive operational information. Public/private distinction is not meaningful across domains.
Hashtags, pinned status, edit history	Operational metadata that adds attack surface without proportional value.
Interactive message elements (buttons, menus)	Interactive elements contain callback URLs and action IDs that reference the source server's internal state. Relaying them would create non-functional UI elements or potential SSRF vectors.
File previews, embedded links, Open Graph data	Preview data may contain fetched content from the source domain's network, which should not be exposed to the receiving domain.

8. User Identity Across Domains

Cross Guard supports two identity models for how inbound messages are attributed to local users. The choice between them is a security/usability tradeoff controlled by the `UsernameLookup` plugin setting.

Model 1: Username Lookup (Default, UsernameLookup=true)

Behavior	Inbound messages are posted as the matching local user if a user with the same username exists on the receiving server.
Advantage	Seamless user experience. Messages appear to come from the "right" person. No visual distinction between local and relayed posts.
Risk	If Remote Domain A has a user "jsmith" and Local Domain B also has "jsmith", messages from Remote jsmith appear as if Local jsmith wrote them. This is username impersonation if the two accounts belong to different people.
Appropriate When	The same identity provider (LDAP, Active Directory, SSO) provisions users on both servers, so a username match implies the same person.

Model 2: Sync Users (UsernameLookup=false, or No Match Found)

Behavior	A synthetic "sync user" is created with a munged username (e.g., `alice.relay-from-a`). All inbound messages from that remote user are posted under this sync user.
Sync User Properties	`Position="crossguard-sync"`, `LastName="(via connName)"`, `Props.CrossguardRemoteUsername=original`
Advantage	Clear visual distinction between local and relayed users. No impersonation risk. The webapp UserPopover component shows "Relayed from: alice (via relay-from-a)" when hovering over the sync user.
Appropriate When	Usernames do not match across domains, or when impersonation risk is unacceptable (recommended for CDS deployments).

Hybrid Behavior

When username lookup is enabled, resolution is per-message. If a matching local user exists, that user is used. If no match is found, a sync user is created for that specific remote username. This

means messages in the same channel may appear from a mix of real local users and sync users, depending on which remote usernames have local matches.

9. Connection Management and Administrative Workflows

Connection Configuration (System Console)

Transport connections are configured in the Mattermost System Console under **Plugins > Cross Guard**. Inbound and outbound connections are configured as JSON arrays. Each connection specifies a provider type (NATS, Azure Queue Storage, Azure Blob Storage, or Azure Service Bus) along with provider-specific settings:

- **name:** Unique identifier, lowercase alphanumeric with hyphens (e.g., `relay-to-server-b`)
- **provider:** Transport provider type (`"nats"`, `"azure-queue"`, `"azure-blob"`, or `"azure-servicebus"`). Provider-specific fields are nested under the corresponding key.

NATS Provider Fields (nested under `"nats"`)

- **address:** NATS server address (e.g., `nats://nats:4222`)
- **subject:** NATS subject to publish/subscribe (must start with `crossguard.`)
- **TLS settings:** `tls_enabled`, `client_cert`, `client_key`, `ca_cert`
- **Auth settings:** `auth_type` (`none`, `token`, or `credentials`), plus type-specific fields

Azure Queue Provider Fields (nested under `"azure_queue"`)

- **queue_service_url:** Azure Queue Storage service URL (e.g., `https://<account>.queue.core.windows.net`)
- **blob_service_url:** Azure Blob Storage service URL, required when `file_transfer_enabled` is true
- **account_name / account_key:** Shared-key credentials. Treated as secrets; see Section 11: Secret Storage and Env-Var Injection for the recommended env-var pattern that keeps these out of `config.json` and the Mattermost configuration database.
- **queue_name:** Name of the Azure Storage queue (user-configurable, created automatically if it does not exist)
- **blob_container_name:** Name of the Azure Blob container for file transfers (user-configurable, created automatically if it does not exist)

Azure Blob Provider Fields (nested under `"azure_blob"`)

- **service_url:** Azure Blob Storage service URL
- **account_name / account_key:** Shared-key credentials. Treated as secrets; see Section 11: Secret Storage and Env-Var Injection.
- **blob_container_name:** Container that holds both batch WAL blobs and deferred file blobs (auto-created if missing)
- **flush_interval_seconds:** Interval at which the outbound WAL buffer is sealed and uploaded as a batch blob. Default 60 seconds, minimum 5.
- **blob_lock_max_age_seconds:** Maximum lease age before a stall is broken by another inbound poller. Default 300 seconds, minimum 30.

Azure Service Bus Provider Fields (nested under `"azure_servicebus"`)

- **connection_string:** SAS connection string for the Service Bus namespace. Treated as a secret; see Section 11: Secret Storage and Env-Var Injection.
- **queue_name:** Pre-created Service Bus queue name or hierarchical path. Queues are NOT auto-created; operators must provision them in the Azure portal or via ARM/Terraform with the desired `LockDuration` and `MaxDeliveryCount`.
- **blob_service_url:** Azure Blob Storage service URL for the file attachment sidecar, required when `file_transfer_enabled` is true
- **blob_account_name / blob_account_key:** Shared-key credentials for the blob sidecar storage account (independent of the Service Bus namespace). Treated as secrets; see Section 11: Secret Storage and Env-Var Injection.
- **blob_container_name:** Blob sidecar container for file attachments (auto-created if missing), required when `file_transfer_enabled` is true
- **max_message_size_bytes:** Maximum outbound payload size. Default 192000; maximum 104857600 (100 MiB, Service Bus Premium tier).
- **blob_poll_interval_seconds:** Inbound poll cadence for the blob sidecar. Default 15, minimum 1.

Configuration changes trigger live reconnection. The plugin closes existing transport connections and opens new ones with the updated settings. No plugin restart is required.

Team Initialization Workflow

1. **Admin runs `/crossguard init-team [connection-name]`** in the team to be linked. If only one connection exists, it is auto-selected. If multiple exist, an interactive dialog with a dropdown

is presented.

2. **Connection linked to team:** The connection is stored in the KV store using an atomic compare-and-set (CAS) operation to prevent race conditions.
3. **Announcement posted:** The bot posts in `#town-square`: "Cross Guard connection outbound:relay-to-b linked to this team by @admin."
4. **Global tracking updated:** The team ID is added to the global initialized teams list for status reporting.

Channel Initialization Workflow

1. **Admin runs `/crossguard init-channel [connection-name]`** in the channel. The slash command requires the team to be initialized first via `/crossguard init-team`. The REST API auto-links the team if needed.
2. **Connection linked to channel:** Stored in KV store via CAS operation.
3. **Visual indicator added:** Channel header is updated with a link emoji prefix for visual indication of relay status.
4. **WebSocket event published:** The webapp receives a notification to update channel indicators in real time.
5. **Announcement posted:** Bot posts a confirmation in the channel.

Inbound Acceptance Workflow

When the first inbound message arrives for an unlinked team or channel, the plugin does not silently create the link. Instead, it presents an interactive prompt to administrators:

1. Bot posts an interactive message with Accept and Block buttons to `#town-square` (for team-level) or the target channel (for channel-level)
2. Only one prompt is created per team-connection pair (idempotent via KV store check)
3. A Team Admin or System Admin clicks Accept: the connection is auto-linked, and the prompt is replaced with a success message
4. If Block is clicked: the prompt state is set to "blocked" and no messages flow until `/crossguard reset-prompt` clears it

5. The same flow repeats at the channel level once the team is accepted

Team Name Rewriting

For environments where the remote and local team names differ (common in multi-domain deployments where each domain has its own naming conventions):

- `/crossguard rewrite-team [connection] [remote-team-name]` creates a mapping from the remote team name to the local team where the command is run
- Stored as a reverse index in KV: `crossguard-rwi-{connName}::{remoteTeamName}` maps to the local team ID
- Conflict detection: the same remote team name cannot map to two different local teams (returns HTTP 409)
- Audit trail: bot posts in `#town-square` showing who set or cleared the rewrite

Teardown Workflow

- `/crossguard teardown-channel` and `/crossguard teardown-team` unlink connections
- Removes the link emoji from the channel header
- Posts a farewell announcement
- Does not cascade: `teardown-team` does not automatically teardown channels within the team

10. Loop Prevention Architecture

The Loop Problem

Relay loops are the most critical failure mode for any message relay system. Without prevention, the following cycle occurs:

1. Server A user posts a message
2. Cross Guard on Server A relays the message to Server B via the transport layer
3. Cross Guard on Server B creates the post locally via the Plugin API
4. This triggers Server B's `MessageHasBeenPosted` hook
5. Without prevention: Server B would relay the message back to Server A via the transport layer

6. Server A receives it, creates it locally, triggering its own hook
7. Infinite loop: messages multiply exponentially

Cross Guard uses four independent mechanisms to prevent this, following a defense-in-depth approach where no single mechanism is solely responsible.

Mechanism 1: crossguard_relayed Post Property (Primary)

- Every inbound post is created with `post.AddProp("crossguard_relayed", true)`
- Every outbound hook checks `post.GetProp("crossguard_relayed") != nil` and skips if present
- This defense applies to post create and update hooks. Delete hooks use a separate deleting flag mechanism, and reaction hooks use sync-user position filtering.

Mechanism 2: Bot User Filtering (Secondary)

- All outbound hooks check `post.UserId == p.botUserID` and skip bot posts
- Prevents bot announcements (init/teardown messages, interactive prompts) from being relayed
- Also provides loop protection if a relayed post were somehow attributed to the bot user

Mechanism 3: Sync User Reaction Filtering (Tertiary)

- Reaction hooks check `user.Position == "crossguard-sync"` and skip sync user reactions
- When an inbound reaction is added by a sync user, the local reaction hook fires but is filtered out
- Prevents reaction loops that would otherwise occur with sync users

Mechanism 4: Delete Flag (Delete-Specific)

- Problem: inbound delete calls `DeletePost()`, which fires the `MessageHasBeenDeleted` hook
- Solution: before deleting, set `SetDeletingFlag(localPostID)` in the KV store
- The outbound delete hook checks `IsDeletingFlagSet(post.Id)` and skips if set
- Flag is cleared after the delete completes
- Prevents infinite delete-relay loops

Defense in Depth

> **Multiple independent mechanisms**
> These four mechanisms operate independently. Even if the `crossguard_relayed` property were somehow lost (e.g., a Mattermost bug stripping props), bot user filtering would catch bot-posted messages. Even if a race condition occurred in the delete flag, the post property check would catch the re-relay attempt. No single failure mode can cause a loop.

11. Transport Security Configuration

NATS Transport Security

TLS Configuration

- Minimum TLS 1.2 enforced (hardcoded `tls.VersionTLS12` in the NATS connection options)
- Connection fields: `tls_enabled` (boolean), `client_cert` (file path), `client_key` (file path), `ca_cert` (file path)
- Client certificate and key must be provided together (validation enforced at configuration time)
- CA certificate enables server certificate verification (the plugin verifies the NATS server's identity)

Authentication Options

Method	Configuration	Description	Suitable For
`none`	No additional fields	No NATS authentication. Anyone who can reach the NATS server can publish and subscribe.	Development and testing only
`token`	`token` field	Bearer token sent on NATS connection. Shared secret between the plugin and the NATS server.	Production environments with simple auth requirements
`credentials`	`username` + `password` fields	Username and password sent on NATS connection. Allows per-client identity on the NATS server.	Production and CDS environments requiring per-client ACLs

Connection Resilience

- Persistent connections with unlimited reconnects (`MaxReconnects = -1`)
- 2-second reconnect wait between attempts
- Disconnect events logged at WARN level, reconnect events at INFO level
- Context-aware cancellation: reconnection attempts are abandoned during graceful plugin shutdown

Outbound Publish Reliability

- 3 retry attempts with exponential backoff (500ms, 1s, 2s)
- Each publish followed by a flush call to confirm delivery to the NATS server
- Messages are dropped (not queued) after retry exhaustion
- Context cancellation is respected during backoff waits

Configuration Validation

- **Connection names:** lowercase alphanumeric with hyphens, unique across all connections (both inbound and outbound)
- **Subject:** must start with `crossguard.` prefix (prevents accidental subscription to unrelated NATS traffic)
- **Auth type:** must be `none`, `token`, or `credentials`
- **Conditional fields:** token required for token auth, username and password required for credentials auth

Deployment Sensitivity Guide

Level	TLS	Auth	mTLS	Dedicated NATS	ACLs
Development	Optional	Optional	No	No	No
Production	Required	Token or credentials	Recommended	Recommended	Recommended
CDS (classified)	Required	Credentials	Required	Required	Required

Azure Transport Security

- **Storage Account Credentials (Azure Queue, Azure Blob):** The Azure Storage connection string (or account name plus shared key) contains the storage account credentials and must be treated as a secret. See Secret Storage and Env-Var Injection below for how to keep it out of `config.json`.
- **Service Bus SAS Connection String (Azure Service Bus):** The Service Bus namespace is authenticated with a SAS connection string of the form `Endpoint=sb://<namespace>.servicebus.windows.net/;SharedAccessKeyName=<rule>;SharedAccessKey=<key>`. The plugin sanitizes `SharedAccessKey=`, `SharedAccessSignature=`, and `sig=` query parameters out of any error messages it logs (`sanitizeServiceBusError()`) so SAS material does not leak into operator-visible logs. See Secret Storage and Env-Var Injection below for how to keep it out of `config.json`.
- **Independent Sidecar Credentials:** When the Azure Service Bus provider has `file_transfer_enabled`, its blob sidecar takes its own storage account credentials separate from the Service Bus namespace credentials. The two stores can live in different Azure subscriptions and have independent access policies. Audit and rotate the two credentials independently.
- **Network Security:** Azure Storage and Azure Service Bus both support Virtual Network service endpoints and private endpoints to restrict access to specific networks.
- **Encryption in Transit and at the Provider:** All Azure Storage traffic uses TLS, and Azure Service Bus uses AMQP over TLS. Data at rest is encrypted by Azure Storage Service Encryption (SSE) and by Service Bus's managed encryption at rest. (See the next subsection for credential storage on the Mattermost side, which is a separate concern.)
- **Access Control:** Use Azure RBAC, shared access signatures (SAS), or scoped Service Bus SAS rules for fine-grained access control in production. Prefer scoped Service Bus SAS rules over the namespace-wide `RootManageSharedAccessKey` so an exposed credential grants only the minimum needed rights.

Secret Storage and Env-Var Injection

Mattermost does not encrypt plugin settings at rest. By default, the connections JSON (`OutboundConnections` / `InboundConnections`), including any embedded NATS tokens or passwords, Azure Storage shared keys, or Service Bus SAS connection strings, is stored as plaintext in `config.json` on disk and in the Mattermost configuration database. Any operator with disk, backup, or DB access can read these credentials.

For CDS and any production deployment, inject the connections JSON via Mattermost's environment-variable substitution instead of saving it through the System Console. Mattermost overrides plugin settings at config-load time from environment variables of the form:

```
MM_PLUGINSETTINGS_PLUGINS_CROSSGUARD_OUTBOUNDCONNECTIONS=<json-array>
MM_PLUGINSETTINGS_PLUGINS_CROSSGUARD_INBOUNDCONNECTIONS=<json-array>
```

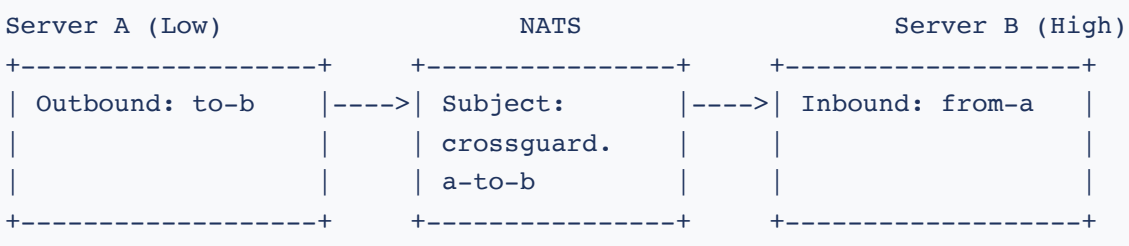
Source these environment variables from a secret store appropriate for your platform: Kubernetes secrets, systemd `LoadCredential=`, HashiCorp Vault, AWS Secrets Manager, Azure Key Vault, or equivalent. The credential then lives only in the Mattermost process environment, not in `config.json` on disk and not in the Mattermost configuration database. Disk and DB backups no longer carry the secret.

Residual exposure with env-var injection: the credential is still readable by anyone with read access to the Mattermost process environment (root on the host, container exec, `/proc/<pid>/environ`, container orchestrator logs that capture the env). System Admins can also still see the in-memory plugin settings through the System Console UI. Combine env-var injection with key rotation and scoped SAS rules so a compromised credential can be rotated quickly and grants only the minimum needed rights.

12. Deployment Topologies

Cross Guard supports multiple deployment topologies depending on the number of servers, the direction of message flow, the transport provider, and the network architecture between domains. Topology diagrams for each transport follow: NATS topologies first, then one diagram per Azure transport provider (Queue Storage, Blob Storage, Service Bus).

Topology 1: Unidirectional Relay (Low-to-High)



```
Messages flow one direction only: Low --> High
```

- Server A has one outbound connection; Server B has one inbound connection
- Same NATS subject configured on both sides
- Messages flow from the low-side to the high-side only
- Use case: low-side collaboration visible on high-side for monitoring or awareness

Topology 2: Bidirectional Relay

```
Server A                          NATS                          Server B
+-------------------+     +-------------------+     +-------------------+
| Outbound: to-b    |---->| crossguard.a-to-b |---->| Inbound: from-a   |
| Inbound: from-b   |<----| crossguard.b-to-a |<----| Outbound: to-a    |
+-------------------+     +-------------------+     +-------------------+

Two subjects, two directions: full two-way conversation flow
```

- Each server has one outbound and one inbound connection
- Different NATS subjects per direction prevents echo at the transport level
- Full two-way conversation flow between domains
- Use case: collaborative communication between domains

Topology 3: Hub-and-Spoke (Multi-Server Broadcast)

```
                              NATS
                         +----------------+
Server A (Hub)           | Subject:       |          Server B
+-------------------+    | crossguard.    |    +-------------------+
| Outbound: to-all  |--->| hub-broadcast  |--->| Inbound: from-hub |
+-------------------+    |                |    +-------------------+
                         |                |
                         |                |          Server C
                         |                |    +-------------------+
                         |                |--->| Inbound: from-hub |
                         |                |    +-------------------+
                         |                |
```

```
                          |                |          Server D
                          |                |   +-------------------+
                          |                |-->| Inbound: from-hub |
                          +----------------+   +-------------------+

One publisher, multiple subscribers on the same subject
```

- Server A publishes; Servers B, C, D subscribe to the same subject
- NATS delivers to all subscribers automatically
- Use case: broadcast from one domain to many receiving domains

Topology 4: Loopback (Same-Server Relay)

```
Server A
+-----------------------------------------------+
| Outbound: loopback  --> NATS -->  Inbound: loopback |
|                                               |
| Team "source-team"                            |
|    Channel: #ops-updates                      |
|         |                                     |
|    Team Name Rewrite:                         |
|    "source-team" --> "dest-team" (local)      |
|         |                                     |
| Team "dest-team"                              |
|    Channel: #ops-updates                      |
+-----------------------------------------------+

Both connections on the same server, team name rewrite routes between teams
```

- Both outbound and inbound connections on the same server, same NATS subject
- Requires team name rewrite to route messages from one team to another
- Use case: internal relay between isolated project teams on the same server

Topology 5: Multi-Hop Relay

```
Server A              NATS 1              Server B              NATS 2              Se
+-----------+     +----------+     +----------------+     +----------+     +----------
| Outbound  |---->| cg.a-b   |---->| Inbound from-a |     |          |     |
|           |     +----------+     | Outbound to-c  |---->| cg.b-c   |---->| Inbound
```

```
+-----------+                    +----------------+     +----------+     +----------
```

```
Server B acts as a relay point. Each hop is independent.
```

- Server B receives from A and re-relays to C
- Each hop is an independent Cross Guard configuration with its own connections
- Messages on Server B are full local posts (editable, deletable, visible to Server B users)
- Use case: cascading classification levels (e.g., Unclassified to Secret to Top Secret)

Azure Queue Storage Topology

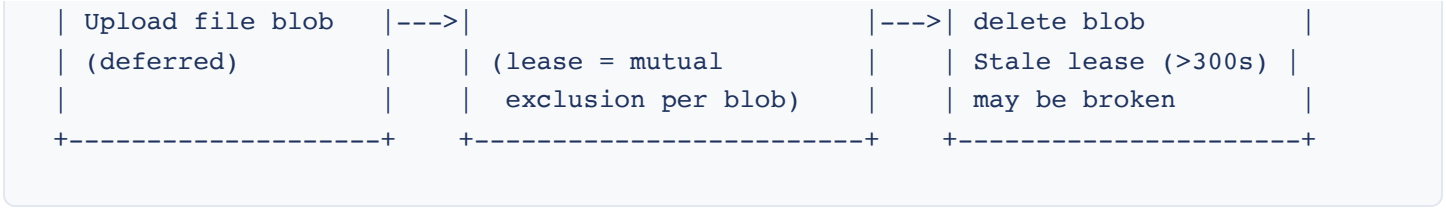
In Azure-native deployments, both Mattermost instances connect to a shared Azure Storage account. The outbound server enqueues messages and uploads file blobs. The inbound server polls the queue and blob container. No additional infrastructure (NATS cluster, JetStream) is required.

```
Server A (Low Side)          Azure Storage Account       Server B (High Side)
+------------------+    +---------------------------+    +------------------+
|                  |    | Queue: (configured)        |    |                  |
| Enqueue message  |--->|                           |--->| Poll queue (5s)  |
|                  |    | Blob: (configured)         |    |                  |
| Upload blob      |--->|                           |--->| Poll blobs (15s) |
+------------------+    +---------------------------+    +------------------+
```

Azure Blob Storage Topology

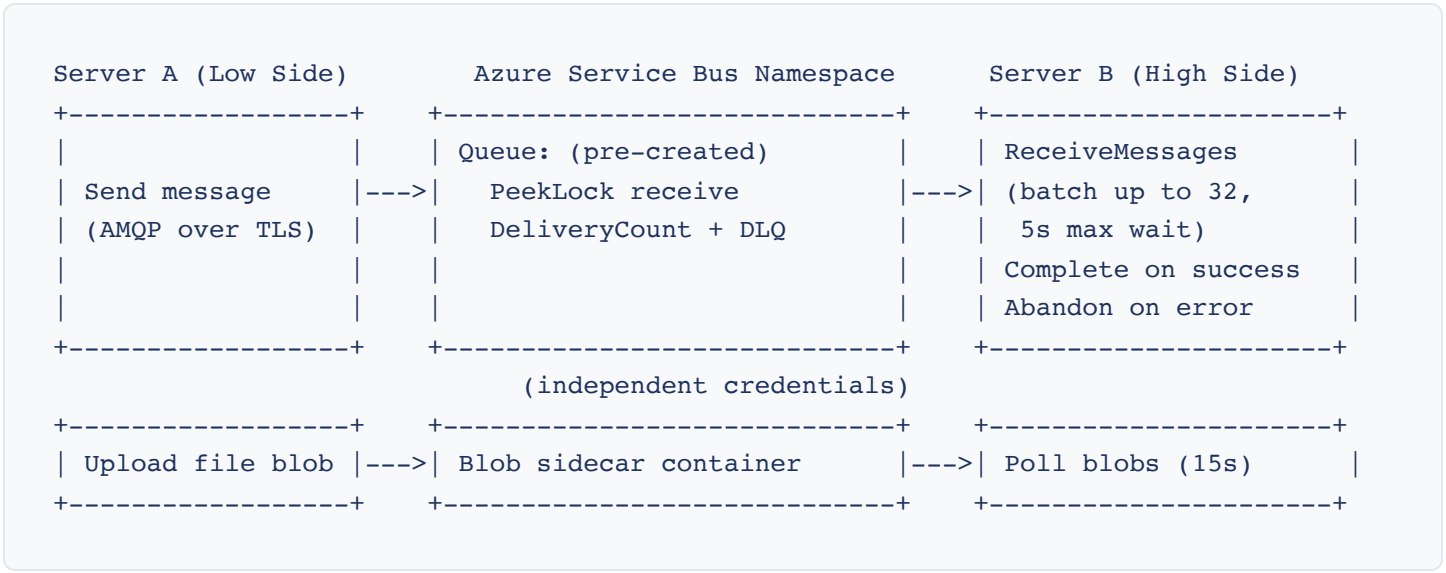
The Azure Blob transport replaces the per-message queue with a batched write-ahead log in a single blob container. Outbound buffers events in memory and every `flush_interval_seconds` seals the buffer and uploads it as a time-ordered batch blob. Inbound takes a short-lived blob lease per batch, processes the messages, then deletes the blob. File attachments live in the same container as deferred file blobs. There is no Azure Queue Storage dependency.

```
Server A (Low Side)          Azure Storage Account       Server B (High Side)
+--------------------+    +-------------------------+    +----------------------+
|                    |    | Container: shared       |    |                      |
| Append to WAL      |    |   - batch blobs (WAL)   |    | List + lease batch   |
| Flush every 60s    |--->|   - deferred file blobs |--->| blob, deliver, then  |
```



Azure Service Bus Topology

The Azure Service Bus transport uses a pre-created Service Bus queue for messages and an optional Azure Blob Storage sidecar container for file attachments. Authentication is via a SAS connection string for the Service Bus namespace and independent storage account credentials for the blob sidecar. Inbound uses PeekLock receive with explicit settle, so failed handlers cause Service Bus to redeliver until `MaxDeliveryCount` is hit and the message is moved to the queue's dead-letter sub-queue.



13. State Management and Persistence

Cross Guard stores all persistent state in Mattermost's built-in plugin KV store. No external database is required. All state is key-value pairs accessed via the Plugin API.

KV Key Schema

Key Pattern	Value	Purpose
`crossguard-teaminit-{teamID}`	`[]TeamConnection` (JSON array)	Which connections are linked to this team
`crossguard-channelinit-{channelID}`	`[]TeamConnection` (JSON array)	Which connections are linked to this channel
`crossguard-initialized-teams`	`[]string` (team IDs)	Global list of initialized teams for status reporting
`pm-{connName}-{remotePostID}`	`string` (local post ID)	Maps remote post IDs to local post IDs for edits/deletes
`crossguard-deleting-{localPostID}`	`true`	Flag set during inbound delete to prevent loop
`crossguard-connprompt-{teamID}-{connName}`	`ConnectionPrompt` (JSON)	Team-level inbound acceptance prompt state
`crossguard-chanprompt-{channelID}-{connName}`	`ConnectionPrompt` (JSON)	Channel-level inbound acceptance prompt state
`crossguard-rwi-{connName}::{remoteTeamName}`	`string` (local team ID)	Team name rewrite reverse index

Caching Layer

An optional caching layer wraps the base KV store to reduce API calls for frequently accessed data:

Cache	Size	TTL	Invalidation Event
Team connections	64 entries	15 minutes	`cache_inv_teaminit`
Channel connections	1024 entries	15 minutes	`cache_inv_chaninit`
Initialized teams list	1 entry	15 minutes	`cache_inv_initteams`
Rewrite index	256 entries	15 minutes	`cache_inv_rwindex`

The cache is cluster-aware: write operations publish invalidation events to all cluster nodes via `PluginClusterEvent`. This ensures cache consistency in multi-node Mattermost deployments.

CAS Operations

List modifications (adding or removing a team connection, channel connection, or initialized team) use atomic compare-and-set (CAS) with 5 retries. This prevents race conditions when multiple admin actions occur simultaneously. All modifications have all-or-nothing semantics (no partial updates).

14. Concurrency Model

Relay Semaphore

The relay semaphore is a buffered channel of capacity 256, shared by both inbound and outbound relay operations. It provides backpressure to prevent unbounded goroutine spawning under high message volume:

- Non-blocking acquisition: if the semaphore is full, the message is dropped with a warning log rather than blocking
- Each relay operation: acquire slot, spawn goroutine, process message, release slot
- The drop-when-full policy ensures the Mattermost server process is never starved of resources by relay operations

File Transfer Semaphore

A separate file transfer semaphore (buffered channel of capacity 32) controls concurrent file upload and download operations. This prevents file transfers from consuming excessive bandwidth or memory:

- Outbound file uploads use non-blocking acquisition: if all 32 slots are in use, the file upload is skipped for that message
- Inbound file downloads use blocking acquisition: the goroutine waits until a slot becomes available before downloading
- This asymmetry ensures inbound files are reliably downloaded while outbound uploads do not block message relay

Goroutine Lifecycle

- All relay goroutines are tracked via a `sync.WaitGroup`
- Context cancellation signals shutdown to all goroutines
- OnDeactivate sequence: cancel context, close inbound subscriptions, wait for all goroutines, close outbound connections
- This ensures no orphaned goroutines remain after plugin deactivation

Connection Pool Protection

Resource	Lock Type	Read Lock (Concurrent)	Write Lock (Exclusive)
Outbound connections	`sync.RWMutex`	Publishing messages	Reconnection on config change
Inbound connections	`sync.RWMutex`	Subscription access	Reconnection on config change
Configuration	`sync.RWMutex`	Reading current config	Config update

NATS Client Concurrency

The NATS Go client is goroutine-safe for publish operations. Subscription callbacks run in separate NATS-managed goroutines. Cross Guard wraps these callbacks with semaphore control to limit the total number of concurrent relay operations across both directions.

15. Operational Monitoring and Troubleshooting

Key Log Messages

Level	Message	Meaning	Action
WARN	"Relay semaphore full, dropping inbound message"	Message volume exceeds processing capacity	Check for message flood or consider scaling

Level	Message	Meaning	Action
WARN	(no log message, connection silently skipped)	Outbound connection is unhealthy (health check failed)	Check transport server health and network connectivity. The connection will be rechecked after 30 seconds.
WARN	"Inbound update: no post mapping found"	Edit/delete arrived for a post that was not previously relayed	Expected if channel was linked after the original post
WARN	"Unknown inbound message type"	Unexpected envelope type received	Investigate possible message injection or version mismatch
ERROR	"Failed to publish to outbound after retries"	Transport unreachable after retry attempts	Check transport server, network, TLS certificates
ERROR	"Inbound post: create failed"	Mattermost API rejected the post creation	Check post content, channel permissions, server health
INFO	"Inbound NATS reconnected"	NATS connection recovered after interruption (NATS provider only; Azure uses polling, so there are no reconnect events)	Normal recovery, verify message flow resumed
INFO	"Outbound NATS reconnected"	NATS connection recovered after interruption (NATS provider only; Azure uses polling, so there are no reconnect events)	Normal recovery, verify message flow resumed

Status Commands

- `/crossguard status`: Shows team and channel linkage state. System Admins see a global overview across all teams and connections. Team members see their team/channel status.

REST API Status Endpoints

Endpoint	Permission	Returns
`GET /api/v1/status`	System Admin	Global status with all teams, connections, and warnings
`GET /api/v1/teams/{id}/status`	Team member	Team-specific status including orphaned connections
`GET /api/v1/channels/{id}/status`	Channel member	Channel-specific status

Connection Testing

`POST /api/v1/test-connection` (System Admin only) tests transport connectivity without affecting running connections:

- Outbound test: publishes a test message and confirms the flush succeeds
- Inbound test: subscribes to the subject and confirms the subscription is active

Sync User Audit

To audit synthetic relay users, query Mattermost users with `Position="crossguard-sync"`. Each sync user's `LastName` shows the connection name (e.g., `(via relay-from-a)`), and `Props.CrossguardRemoteUsername` shows the original remote username.

16. Comparison with Alternative Approaches

Capability	Cross Guard (NATS / Azure)	Shared Database	Direct API	Email Bridge	Webhook Relay
Server independence	Full	None (shared DB)	Partial	Full	Full
Network isolation	Yes (NATS in DMZ / Azure private endpoint)	No (DB access)	No (HTTP access)	Yes	No (HTTP access)

Capability	Cross Guard (NATS / Azure)	Shared Database	Direct API	Email Bridge	Webhook Relay
Real-time messaging	Yes (sub-ms NATS / 5-15s Azure)	N/A	Yes	No (minutes)	Yes
Edit/delete sync	Yes	N/A	Possible	No	Manual
Reaction sync	Yes	N/A	Possible	No	No
Thread support	Yes (root_id mapping)	N/A	Possible	No	No
Admin opt-in control	Team + channel level	N/A	N/A	N/A	Per-webhook
Transport security	TLS 1.2+, mTLS, ACLs (NATS) / TLS always, RBAC (Azure)	DB TLS	HTTPS	SMTP TLS	HTTPS
Message authentication	NATS auth / Azure connection string	DB auth	API auth	DKIM/SPF	Shared secret
Resilience	Auto-reconnect	DB failover	Retry logic	Store-and-forward	Manual
Data minimization	Text + optional files (disabled by default, per-connection toggle, type filtering). Azure blobs deleted after processing.	Full DB access	Full API access	Full email	Configurable
Audit trail	Plugin logs + post props	DB audit log	API logs	Mail logs	Webhook logs

17. CDS Deployment Checklist

> **Hardened Configuration**

> The following checklist is recommended for Cross Guard deployments in classified or sensitive environments. For a detailed STRIDE threat analysis of each risk, see the Threat Model.

1. **Enable `RestrictToSystemAdmins`**: Lock all connection management commands to System Admins. Prevents Team Admins and Channel Admins from enabling relay without organizational approval.

2. **Disable `UsernameLookup`**: Force all inbound messages through synthetic sync users. Eliminates impersonation risk (T-SPOOF-1).

3. **Enable TLS with mutual certificate authentication (NATS)**: For NATS connections, configure `tls_enabled`, `client_cert`, `client_key`, and `ca_cert`. Ensures both the plugin and the NATS server verify each other's identity. For Azure connections, TLS is always enabled by the Azure endpoint.

4. **Deploy a dedicated transport endpoint**: For NATS, do not share the NATS server with other applications. For Azure, use a dedicated Storage account for Cross Guard traffic. Reduces the risk of message content exposure (T-INFO-1).

5. **Configure transport-level access control**: For NATS, restrict publish and subscribe permissions per subject per client via NATS ACLs. For Azure, use Azure RBAC or SAS tokens to restrict access to the storage account. Ensure each plugin instance can only access the resources it is configured for.

6. **Use separate subjects per relay direction (NATS)**: For NATS, configure different subjects for each direction (e.g., `crossguard.a-to-b` and `crossguard.b-to-a`). Prevents transport-level echo. For Azure, use separate queues or storage accounts per direction.

7. **Place the transport endpoint in a controlled network segment**: For NATS, deploy the server in a DMZ, enclave, or guard network between the two domains. Firewall rules should allow only NATS traffic (port 4222) between segments. For Azure, use private endpoints or VNet service endpoints to restrict storage account access to authorized networks.

8. **Monitor plugin logs**: Watch for anomaly indicators: "Relay semaphore full" (message flood), "Unknown inbound message type" (possible injection), failed resolve warnings (misconfiguration or probing).

9. **Audit sync users periodically**: Query users with `Position="crossguard-sync"` to detect unexpected user creation that might indicate unauthorized relay activity.

10. **Enforce `crossguard.` subject prefix**: This is validated in configuration. All NATS subjects must start with `crossguard.` to prevent accidental subscription to unrelated NATS traffic.

11. **Document and review team name rewrite rules**: Maintain an inventory of all rewrite mappings. Rewrites change message routing and should be reviewed as part of the security configuration.

12. **Test connections regularly**: Use the admin console test button or the `POST /api/v1/test-connection` endpoint to verify transport connectivity without affecting running connections.

13. **Review the Threat Model**: The Threat Model page provides a detailed STRIDE analysis of 14 identified threats with severity ratings, code paths, mitigations, and residual risks.

18. References

Overview

Getting started, key concepts, permission model, and federation overview.

Slash Command Reference

All 9 slash commands with arguments, permissions, and examples.

Admin Configuration Guide

Transport connection setup, relay settings, and example configurations.

REST API Reference

All REST endpoints with request/response examples and authentication details.

Threat Model

STRIDE analysis, trust zones, 14 identified threats, and CDS deployment recommendations.